

Build an Agentic Telegram Teaching Assistant

Local LLM + Telegram Bot + Slide Understanding + Web Research + Email Delivery

Assignment in one sentence. Build a Telegram bot that accepts lecture slides, runs a local LLM through vLLM or llama.cpp, creates a useful lesson-planning package, researches supporting material online, and emails the result after user approval.

Course	Natural Language Processing, American University of Armenia (AUA)
Format	Individual homework
Due date	TBD
Inspiration	ArmLLM agents materials: https://github.com/osoblanco/ArmLLM/tree/main/2025/Agents

1. Goal

The goal is to build a small but complete LLM application: a Telegram bot that can receive lecture slides, understand the topic, prepare a teaching plan, find useful external links, and send the generated package by email. The focus is on system design, tool integration, and reliability - not on using the largest possible model.

2. Expected user flow

- User opens the bot and receives a short help message.
- User uploads lecture slides, preferably PDF. PPTX support is a bonus.
- User provides lecture duration, target audience, output language, and recipient email.
- Bot parses the slides, calls the local LLM, searches the web, drafts the package, and shows a preview.
- Bot sends the email only after the user confirms the preview.

3. Minimal commands

Command	Expected behavior
/start	Introduce the bot and show a small example.
/help	List commands, arguments, and basic limitations.
/plan	Run the slide-based lesson planning workflow.
/research	Return a compact list of supporting web resources.
/status	Show uploaded files, current state, and errors.
/send	Send the latest approved report by email.

4. Architecture

You may use LangGraph, LlamaIndex, Haystack, a ReAct-style loop, or a simple custom state machine. A simple finite-state design is completely acceptable if it is clear and debuggable.

```

Telegram Bot API -> Session State -> Agent Orchestrator
                        |
                        +-> Slide Parser / Retriever
                        +-> Web Search
                        +-> Email Sender
                        +-> Logger / Status
  
```

Local LLM backend: vLLM OpenAI-compatible server or llama.cpp server

5. Local LLM backend

- Use vLLM if you have GPU access, or llama.cpp with a GGUF model for CPU/smaller GPU setups.
- Wrap the backend behind a function such as `generate(messages, temperature, max_tokens)`.
- A cloud model may be used only as an optional comparison, not as the main submitted system.
- Document the model name, quantization if any, context length, and exact command used to run the server.

6. Agent workflow

Step	Minimum output
Slide ingestion	Extract slide text with slide/page references and produce a short summary.
Concept map	List the main concepts and likely prerequisites.
Teaching plan	Create a timed plan with objectives, examples, exercises, and recap.
Web research	Find at least 3 useful external resources with URLs and short justifications.
Revision	Check that the plan is realistic, clear, and mostly grounded in the slides.
Email	Prepare a professional email body and send it after confirmation.

7. Minimum technical expectations

Keep this lightweight. Your agent should expose four practical tools: slide parsing/retrieval, web search, email sending, and logging/status. The final package should include a title, audience, duration, learning objectives, a timed plan, at least one exercise, useful links, and a short email body.

Basic reliability expectations: do not commit secrets; use environment variables for tokens; show an email preview before sending; handle bad files or failed web search without crashing. Avoid over-engineering - clear behavior matters more than a large guardrail framework.

8. Recommended repository structure

```
agentic-teaching-bot/  
  README.md  
  .env.example  
  bot.py  
  llm_backend.py  
  agents/  
    orchestrator.py  
    prompts.py  
  tools/  
    slides.py  
    web_search.py  
    email.py  
  examples/  
    sample_slides.pdf  
    sample_output.md
```

9. Individual deliverables

- GitHub repository with setup instructions and `.env.example`.
- Short demo video or live demo showing upload -> plan -> research -> preview -> email.
- One sample generated teaching package from real lecture slides.
- Design report of 2-4 pages explaining architecture, model choice, prompts, tools, and limitations.
- A small evaluation section with at least 3 test cases.

10. Evaluation

- Functional test: upload slides, generate plan, research, preview, send email.

- Grounding check: mark a few claims as coming from slides or web sources.
- Failure test: show behavior for a bad file, missing email, or failed web search.
- Latency note: report approximate runtime and hardware/backend used.

11. Grading rubric: 100 points

Category	Points	What earns full credit
Telegram bot UX	10	Clear flow, usable commands, status messages, preview before email.
Local LLM backend	12	Clean vLLM or llama.cpp integration, documented and swappable.
Slide understanding/RAG	15	Good extraction, chunking/retrieval, slide references.
Agent workflow	18	Multi-step workflow with tools, state, revision, and logs.
Web research	10	Useful URLs, no fake references, sensible summaries.
Email integration	8	Secure credentials, professional email, explicit confirmation.
Output quality	15	Useful objectives, realistic timing, clear exercises and writing.
Evaluation/report	8	Meaningful tests, examples, limitations, and analysis.
Code quality	4	Readable, modular, documented, no secrets committed.

12. Bonus ideas

- Support Armenian or bilingual Armenian/English plans.
- Generate a PDF or Markdown attachment.
- Add a simple dashboard for inspecting agent traces.
- Compare vLLM vs llama.cpp on latency and output quality.

Academic integrity

This is an individual homework. You may discuss high-level ideas, but the submitted code, prompts, report, and demo must be your own. Open-source frameworks and AI-generated code are allowed, but you must cite external sources and understand what you submit.

Final checklist. Clean checkout works; README explains setup; .env.example is present; no secrets are committed; sample output is included; demo shows the full workflow; email sending requires confirmation.